EXAM PLAYBOOK (23 MC questions, A to E)

- **Compute the output yourself first**, then look at options. In the sample exam, "None of the above" (E) was correct 4 of 23 times (Q3, Q7, Q8, Q9) because the four listed options were near-miss decoys.
- `print(a, b)` separates with a space; `print(list(x))` shows brackets. Only the **last expression** in a cell auto-displays; earlier lines show nothing without `print`.
- Read the question's data carefully: **NaN, duplicates, sort order, inplace, and which axis** (rows vs columns) decide most traps. Full trap list on the back.

COMMON ERRORS (what raises what)

- `IndexError` list index out of range. `KeyError` missing dict key or column label. `TypeError` wrong type mix (`"a" + 1`, averaging text, missing parentheses in a mask). `ValueError` bad length or ambiguous truth (`and`/`or` on Series, wrong-length column). `SyntaxError` invalid code (`df.Net Income`). `NameError` variable not defined yet (run cells in order).

OUTPUT AND DATA TYPES

Every value is an **object** with a type. `type(x)` returns its class.

Type	Literal	Notes
`int`	`10`	whole number
`float`	`10.3`	decimal; int op float gives float
`bool`	`True False`	capital first letter; `type(True)` is bool
`str`	`"Hello" 'Hi'`	text. Quotes make it text: `type("250")` is **str**, not int
`list`	`[1, "a"]`	ordered, changeable (mutable)
`dict`	`{"k": 1}`	key: value pairs, keys unique

MATH OPERATORS

Op	Name	Examples
`+ -`	add, subtract	`11+5` = 16
`* **`	multiply, power	`3**4` = 81, `5**2` = 25
`/`	true division	**always float**: `16/2` = 8.0, `5/2` = 2.5
`//`	floor division	`5//2` = 2, `-5//2` = **-3** (rounds toward minus infinity, not zero)
`%`	remainder (modulo)	`18%5` = 3, `5%2` = 1

Precedence: `**`, then `* / // %`, then `+ -`; parentheses override. Q22: `halve(x)` returns `x/2`, so `halve(16)`, `halve(5)` print `8.0 2.5`. Q23: only `5/2` gives 2.5 (`5%2`=1, `5//2`=2, `5**2`=25).

BOOLEANS AND COMPARISONS

Comparison operators: `==` equal, `!=` different, `> < >= <=`. Result is a bool. `=` **assigns**, `==` **compares**. String comparison is **case-sensitive**: `"Python" == "python"` is False (Q21); `"abc" != "ABC"` is True.

Boolean operators: `and` (both True), `or` (at least one True), `not` (flips). Precedence: `not`, then `and`, then `or`. With `1 > 2` (False) and `"abc" != "ABC"` (True): `and` gives False, `or` gives True, `not (...or...)` gives False.

STRINGS

- Immutable text. A quote inside a string: switch quote type `'She said: "Hi!"'` or escape `"She said: \"Hi!\""`.
- Methods **return a new string** (original unchanged): `.upper()` `.lower()` `.title()` `.count("l")` `.replace("o","u")`. Also `len(s)`, indexing `s[0]`, slicing `s[1:3]`.
- Join with `+` (str + str only; str + int is a TypeError): `m1 + " " + m2`. **f-string**: `f"{m1} {m2}"`, braces evaluate expressions.

VARIABLES

- `name = value` stores a value (case-sensitive, no spaces, cannot start with a digit). Reassigning overwrites. Type a name alone to display it.
- **Aliasing:** `y = x` for a list makes both names point to the **same** list, so changing x changes y. Use a copy to separate them.

LISTS (ordered, mutable, square brackets)

- **Index** is 0-based. Negative counts from the end: `-1` last, `-len` first. `colors = ['red','green','blue','yellow']`: `colors[-3]` = 'green' (Q10). Out of range raises IndexError.
- **Slice** `lst[start:stop]`: start included, **stop excluded**; blanks default to ends. `[:3]` first three, `[1:]` from second on, `[1:3]` 2nd and 3rd, `[-2:]` last two. A slice is a new list.

Task	Code (what happens)
add at end	`lst.append("Canada")` (in place)
add at position	`lst.insert(0, "Canada")` (in place; shifts the rest)
join lists	`a + b` (new list); nested: `[a, b]` gives list of lists
remove by value	`lst.remove("India")` (first match only)
remove by index	`lst.pop(i)` (returns the item; default last), `del lst[1]`
sort	`lst.sort()` in place, **returns None** (`x = lst.sort()` makes x None); `lst.sort(reverse=True)` descending; `sorted(lst)` returns new list
update	`lst[0] = 1000`
copy	`lst[:]` or `lst.copy()` (independent, shallow); `y = x` is NOT a copy

```
x = [5, 6]; y = x.copy(); x.append(7); print(y)   # [5, 6]  (Q1)
y = x (no copy) instead would print [5, 6, 7]
```

DICTIONARIES (curly braces, key: value)

- Look up by **key**, not position. `d = {"name":"Frank","age":26}`; `d["name"]` is "Frank"; a missing key raises KeyError.
- `d.keys()`, `d.values()`, `d.items()` (key, value pairs). `"name" in d` tests keys.
- Add or change: `d["height"] = 1.7`; `d.update({"height":1.8, "languages":["English","Spanish"]})` changes existing keys and adds new ones. Values can be lists.
- Remove: `d.pop("height")` (returns value), `del d["languages"]`, `d.clear()` empties it. Copy: `d.copy()`.

CONTROL FLOW

```
if age >= 18:      print("adult")
elif age >= 13:    print("teenager")
else:              print("kid")
```

Colon plus indentation required. **Only the first True branch runs**; `age = 18` prints adult only (Q18). Membership: `if "Brazil" in countries:`.

```
for c in countries: print(c, len(c))     # item by item
for i, c in enumerate(countries): ...    # index AND item
for k, v in my_data.items(): print(k, v) # dict loop
for n in range(1, 10): ...  # 1..9;  range(1,10,2) gives 1,3,5,7,9;
range(10) gives 0..9
```

`range(stop)`, `range(start, stop)`, `range(start, stop, step)`: **stop is excluded**. Wrap in `list(range(...))` to see values.

FUNCTIONS

```
def sum_values(a, b):
    x = a + b
    return x
sum_values(2, 3)     # 5
```

Body runs only when called. Parameters are names in `def`; arguments are values passed in. No `return` gives `None`. Built-ins: `len max min sum round(x, nd) type range sorted print`.

SETUP AND VOCABULARY

```
import pandas as pd
import numpy as np
```

- **DataFrame** = 2D table (rows x columns). **Series** = one column (1D, with an index). **Index** = row labels (default 0 to n-1). **Columns** = column labels (names). Each column has one dtype (`int64`, `float64`, `object` for text, `bool`, `datetime64`).

CREATING A DATAFRAME

Source	Code and rule
numpy array	`data = np.array([[1,4],[2,5],[3,6]])` then `pd.DataFrame(data, index=['row1','row2','row3'], columns=['col1','col2'])`
list of lists	`[[7,8,9],[10,11,12]]`: **each inner list is one ROW**, so with 3 column names shape = (2, 3) (Q2)
dict	**keys = column names, values = column lists** (equal length): `pd.DataFrame({'States': states, 'Population': pop})`
csv / URL	`pd.read_csv('file.csv')` or a URL

`index=` needs one label per row; `columns=` one name per column. Always display the result to check it before moving on.

INSPECTING A DATAFRAME

- **Attributes** (no parentheses): `.shape` gives (rows, cols) tuple; `.index` row labels; `.columns` column **labels** (not the data, Q15); `.dtypes` type per column.
- **Methods** (parentheses): `.head(n)` first n rows, `.tail(n)` last n (default 5); `.info()` columns, **non-null counts**, dtypes, memory (missing = rows minus non-null); `.describe()` count, mean, std, min, 25%, 50%, 75%, max for **numeric columns only** (4 columns with 2 numeric gives `describe().shape[1]` = 2, Q14); `describe(include='all')` adds text columns.
- **Built-ins on a DataFrame:** `len(df)` = number of rows; `type(df)` = DataFrame; `max(df)` / `min(df)` act on the column **names**, so they are useless (use `df['c'].max()`); `max(df.index)` and `min(df.index)` work. `round(df, 2)` returns a new frame (numeric columns only, original unchanged). `'col' in df` tests column labels, not values.

SELECTING COLUMNS

Code	Returns	Note
`df['Dividends']`	Series	1D
`df[['Dividends']]`	DataFrame	double brackets, one column
`df[['Ticker','Dividends']]`	DataFrame	columns in the order you list (reorders)
`df.Dividends`	Series	dot syntax, limited

- **Dot syntax fails** when the name has a space (`df.Net Income` is a **SyntaxError**, Q16), clashes with an existing attribute or method (`df.count`, `df.size`, `df.index`), or when creating a column. Brackets always work.
- A selected Series has its own `.index` (same row labels) and `.head()`. Rows by label: `df.loc['row1']`; `df.loc[[labels]]` returns several full rows.

ADDING COLUMNS

Method	Behavior
`df['rating'] = 70`	scalar **broadcast** to every row; assigning an existing name **overwrites** it
`df['r'] = arr`	array, list or Series: length **must equal** number of rows (else ValueError)
`np.arange(n)`	0 to n-1
`np.random.randint(1, 100, size=n)`	random ints, **high excluded**: values 1 to 99, never 100
`np.random.uniform(1, 100, size=n)`	random floats between 1 and 100
`df = df.assign(s1=a, s2=b)`	returns a **new** frame with several columns in one call, chainable, does not touch the original unless you reassign
`df.insert(1, 'test', s1)`	**in place**, returns None; places the column at position 1 (0-based) and shifts others right; error if the name already exists

- **Vectorized math** works elementwise on whole columns: `df['ret'] = (df['p1'] - df['p0']) / df['p0'] * 100` gives [20.0, 25.0] for p0 = [50, 80], p1 = [60, 100] (Q11). Divide by 0 gives `inf`; missing values propagate as NaN. Operations align on the index.

COLUMN STATISTICS (on a Series)

- `.sum()` `.count()` `.mean()` `.median()` `.std()` `.max()` `.min()` `.describe()`. NaN is skipped by default. `.count()` = non-missing values; `len()` = all rows.
- Example: `df['Payout'] = df['Dividends'] / df['Net Income']`, then `df['Payout'].describe()`.

VALUE COUNTS

- `df['Sector'].value_counts()` counts rows per distinct value, **sorted descending**, NaN excluded by default (`dropna=False` to include).
- `normalize=True` gives proportions (sum to 1); chain `.round(3)`. On a frame: `df.value_counts('Ticker')`.

MISSING VALUES

- `.isna()` returns a boolean Series (True where missing); `.notna()` is the opposite. `.isna().sum()` counts missing because True counts as 1. `.info()` non-null counts show gaps too.
- Always `count() + isna().sum() = len`. Q5: 8 funds, 3 NaN fees, so `df['Fee'].count()` = 5 and `isna().sum()` = 3, printed `5 3`.
- `.fillna(0)` **returns a new Series**; assign back: `df['Dividends'] = df['Dividends'].fillna(0)`. `.dropna()` removes rows with NaN. After filling with 0, `describe()` count rises to all rows and the mean falls. Test missing with `isna()`, never `== np.nan`.

SORTING

- `df.sort_values('Total Assets')` ascending by default; `ascending=False` for descending. NaN go last.
- Several columns: `sort_values(['Sector','Total Assets'], ascending=[True, False])` gives sector A to Z, then assets large to small. A single `ascending=False` applies to **all** columns.
- Returns a **new** frame. `inplace=True` changes df and returns **None**; never write `df = df.sort_values(..., inplace=True)` (df becomes None).
- `df.sort_index()` sorts by row labels; `ascending=False` reverses.

INDEX: SET, RESET, BUILD A KEY

- `df.set_index('key', inplace=True)` moves a column into the index, so columns drop by one: shape (2, 3) becomes **(2, 2)** (Q19). `df.reset_index()` moves it back and restores 0 to n-1.
- Build a key: `df['year_ticker'] = df['Fiscal Year'].astype(str) + '_' + df['Ticker']`. `.astype(str)` is needed, since int + str is a TypeError.

RENAMING

- Columns: `df.rename(columns={'Payout':'Payout Ratio'})`; several at once: `{'EOY Stock Price':'P', 'Cash, ...':'Cash'}`. Rows: `df.rename(index={'z':'last'}, inplace=True)`.
- Returns a **new** frame unless `inplace=True`. Labels **not in the mapping stay unchanged**: index ['x','y','z'] becomes `['x','y','last']` (Q17).

FILTERING = BOOLEAN MASK

- A condition on a column makes a bool Series; `df[mask]` keeps rows where True. `df[df['Ticker'] == 'AAPL']`. Check with `.value_counts('Ticker')`.
- Filtering removes **rows only**: columns unchanged and row labels keep their original numbers. 3 of 5 rows are XOM in a 3-column frame: shape **(3, 3)** (Q6).
- Strings: `.str.startswith('A')`, `.str.endswith()`, `.str.contains()`. Numbers: `df[df['EOY Stock Price'] > 2000]`.
- **Combine:** `&` and, `|` or, `~` not, **each condition in its own parentheses**.

```
df[(df['Price'] > 100) & (df['Tier'] == 'Big')]       # correct (Q13)
df[(df['Ticker'].str.startswith('A')) & ((df['Price'] > 2000) |
(df['Tier'] == 'Big'))]
```

- **Errors:** `and` / `or` raise ValueError (ambiguous truth value). Missing parentheses raise TypeError because `&` binds tighter than `>` and `==`. A comma inside the brackets also fails. Parentheses set meaning: `A & (B | C)` differs from `(A & B) | C` (Q20 gives ['AA','AB']).
- **Stored masks:** `f1 = df['Ticker'].isin(['AAPL','TSLA'])`, `f2 = df['Fiscal Year'] == 2023`, then `df[f1 & f2]`.
- `.isin(list)` = membership (many ORs); `~df['Ticker'].isin([...])` = not in list.
- `.query("Dividends > 1e10")` is the same filter as a string. Supports `and`/`or`, `@var` for Python variables, quotes for text values, and backticks around column names with spaces:

```
df.query("`Net Income` > 0 and Sector == 'Energy'")
df.query("Dividends > @cutoff")      # @ pulls in a Python variable
```

np.where: TWO-WAY CONDITIONAL COLUMN

```
med = df['Market Cap'].median()
df['size_tier'] = np.where(df['Market Cap'] > med, 'Big', 'Small')
```

`np.where(cond, if_true, if_false)` returns an array, which you assign to a column. About half the rows fall on each side of the median. Check with `df['size_tier'].value_counts()`. For 3 or more categories, nest `np.where` calls or use `np.select`.

DUPLICATES

Code	Marks as True
`duplicated('c')` or `keep='first'`	every repeat after the first occurrence (first stays False)
`keep='last'`	every row except the last occurrence
`keep=False`	**all** rows whose value appears more than once

- `df[~df.duplicated('Ticker')]` keeps the first row per ticker. `df[~df.duplicated('Ticker', keep=False)]` keeps values that appear **exactly once**.
- `drop_duplicates(subset='Sector', keep='first'|'last')` is the shortcut. Without `subset`, whole rows must match.
- **Smallest/biggest per group:** (1) filter one year, (2) sort ascending by group then by size, (3) `drop_duplicates('Sector')` keeps the **smallest** per sector; `keep='last'` keeps the **biggest**. Order after sorting decides which row survives.

unique, nunique, sample

- `.unique()` returns an array of distinct values in order of appearance, **including NaN**. `.nunique()` counts distinct values and **ignores NaN**. Without NaN, `nunique()` = `len(unique())` (Q7 prints `3 3`).
- `df['Ticker'].sample(10)` random values; `df.sample(10)` random rows; `df.sample(frac=0.1)` random 10 percent; `random_state=42` makes it reproducible.

GROUPBY = SPLIT, APPLY, COMBINE

- `by = df.groupby('Sector')` is a lazy GroupBy object (no result yet). `len(by)` number of groups; `by.groups.keys()` group names; `by.get_group('Energy')` sub-frame of that group.
- Result rows are in **sorted key order** (Energy before Tech) and the key becomes the **index** (`as_index=False` or `.reset_index()` to get a column). NaN keys are dropped.
- `df.groupby('Sector').mean(numeric_only=True)`: text columns cannot be averaged, so without `numeric_only=True` you get a TypeError. Select columns first to avoid it: `['Net Income'].mean()` gives a Series; `[['Net Income','Market Cap']].mean()` gives a DataFrame. `['Net Income'].describe()` gives stats per group.
- **size vs count:** `size()` = rows per group **including NaN**; `count()` = non-missing values in a column. Q3: Tech has 3 rows, 1 dividend; Energy 2 and 2. `list(size())` = [2, 3], `list(['Dividends'].count())` = [2, 1].
- **agg with a list** `['count','mean','median','max']`: columns are named after the **functions** (Q12: `['min','max']`, shape (3, 2) for 3 desks). **agg with a dict** `{'Profit':'mean','Cap':'max'}`: each column keeps its **own name** and gets its own function (Q4: `['Profit','Cap']`, [30.0, 20.0], [150, 300]).
- **Two keys:** `df.groupby(['Sector','Fiscal Year'])['Net Income'].mean()` gives a MultiIndex Series. `.unstack()` pushes the innermost level into columns (sector rows, year columns). `pd.pivot_table(df, index='Sector', columns='Fiscal Year', values='Net Income', aggfunc='mean')` does both in one call.
- **transform:** `df.groupby('Sector')['Net Income'].transform('mean')` returns a same-length Series (group stat repeated on every row, no shrinking). Use it for `df['vs Sector'] = df['Net Income'] / df['Sector Mean NI']`.
- **idxmax:** `best = df.groupby('Sector')['vs Sector'].idxmax()` gives the row **label** of each group max; `df.loc[best]` fetches those full rows (`idxmin` for the minimum).

Job	Tool
stat per group	`groupby('S')['c'].mean()`
many stats	`.agg(['count','mean','max'])`
different stat per column	`.agg({'a':'mean','b':'max'})`
grid / wide	`.unstack()` or `pivot_table(...)`
group stat on every row	`.transform('mean')`
row of the group max	`.idxmax()` then `df.loc[...]`

TRAP CHECKLIST (from the sample midterm)

- **Copy vs alias:** `.copy()` is independent (Q1). **Inner list = row**, dict key = column (Q2).
- **Shape is (rows, cols):** filtering changes rows, `set_index` removes a column, selecting columns changes cols.
- **NaN:** `count` skips it, `size` and `len` do not; `describe()` numeric only; `nunique` ignores it.
- **Arithmetic:** `/` is float, `//` floors, `%` remainder. **Off-by-one:** `-1` is the last index; `range`, slices and `randint` all **exclude the stop/high value**.
- **inplace=True returns None;** list `sort()` returns None too.
- **Dot syntax** breaks on spaces. **Masks:** `&` and `|` need parentheses; `and` / `or` fail on Series.
- **rename:** unlisted labels stay. **agg:** list gives function names as columns, dict keeps column names.

SAMPLE MIDTERM ANSWER KEY

1 D, 2 B, 3 E, 4 D, 5 D, 6 B, 7 E, 8 E, 9 E, 10 B, 11 D, 12 C, 13 C, 14 A, 15 C, 16 D, 17 C, 18 B, 19 D, 20 C, 21 D, 22 B, 23 A.